Exercise 05: Responsible AI Usage Report

- **Course:** Computational Thinking (CS252 - Class 25A01)
- **Student ID:** 25125048
- **Student Name:** Lý Trần Thái
- **Topic:** Responsible AI Usage in Computing & Software Engineering
- **Submission Date:** October 17, 2026

---

1. Introduction: The Imperative for Responsible AI

Artificial Intelligence (AI), particularly Large Language Models (LLMs) and intelligent coding assistants, has rapidly transformed from experimental research into an indispensable cognitive tool in modern computing. In both academic and industry settings, AI enhances productivity, accelerates prototyping, and democratizes technical knowledge.

However, as articulated in course guidelines and ethical frameworks such as VNPT AI's *Responsible AI* doctrine, AI must not be treated as an autonomous oracle or a shortcut to bypass authentic learning. When students use AI merely to bypass intellectual effort or blindly copy code without comprehension, they undermine their own foundational problem-solving abilities and introduce severe risks—such as hallucinations, security vulnerabilities, copyright violations, and flawed logic.

Responsible AI requires using AI deliberately as an intellectual amplifier while retaining absolute human agency, critical scrutiny, and personal accountability for every deliverable submitted.

---

2. Personal Experience: Have You Used AI Before?

Yes, I have actively integrated AI tools into my daily learning and software development workflow over the past two years.

AI Systems Utilized

- **Conversational LLMs:** ChatGPT (GPT-4o), Claude 3.5 Sonnet, and Google Gemini.
- **Developer Assistants:** GitHub Copilot, Cursor, and Antigravity AI pair-programming agents.
- **Specialized Utilities:** DeepL for precise multilingual translation, and Perplexity for technical citation discovery.

Primary Use Cases & Contexts

1. **Conceptual Understanding & Mental Models:** When learning unfamiliar computer science concepts (such as dynamic programming state transitions, graph traversal edge cases, or memory management in low-level systems), I use LLMs to explain the underlying mechanics with diverse analogies and concrete diagrams.
2. **Boilerplate & Test Generation:** Writing repetitive scaffolding code (e.g., standard data class definitions, unit test suites with parameterized test cases, mock HTTP fixtures).
3. **Systematic Debugging & Diagnostic Explanations:** Pasting compiler tracebacks or subtle runtime exceptions to quickly diagnose the root cause and understand the underlying language specification or

edge conditions.
4. **Documentation & Formatting:** Structuring markdown documentation, generating clean comparison tables, and ensuring adherence to academic formatting guidelines.

---

3. Practical Experiences & Key Tips for Using AI Effectively

Drawing from practical software development and coursework, I have developed a set of core principles and actionable tips for responsible and high-impact AI usage:

Tip 1: Treat AI as a Co-Pilot, Never the Pilot (Human-in-the-Loop)

- **The Principle:** You are the author, architect, and ultimate decision-maker; the AI is simply an interactive assistant.
- **The Practice:** Never commit or submit a single line of code generated by an AI that you cannot explain line-by-line, defend during an oral evaluation, or reproduce under exam conditions. If an AI suggests an algorithm, walk through the logic manually with pen and paper or write test assertions to verify its invariant properties.

Tip 2: Provide Rich Context & Explicit Constraints (Prompt Engineering)

- **The Principle:** Vague prompts produce generic, superficial, or hallucinated responses. High-quality answers require precise context.
- **The Practice:** Always specify:
  - The exact programming language, compiler version, and runtime environment.
  - Required time and space complexity constraints (e.g., "*Design an algorithm running in $O(n \log n)$ time with $O(1)$ auxiliary space*").
  - Explicit edge cases (e.g., empty arrays, integer overflows, negative indices, cyclic graphs).
  - Anti-patterns to avoid (e.g., "*Do not use global variables; use pure functions*").

Tip 3: Verify Against Authoritative Sources & Empirical Tests (Anti-Hallucination)

- **The Principle:** LLMs generate probable text sequences, not guaranteed mathematical truths. They can hallucinate nonexistent API methods, incorrect library syntax, or subtle off-by-one errors with high grammatical confidence.
- **The Practice:** Cross-reference any library recommendation against official documentation (such as docs.python.org or MDN Web Docs). Write comprehensive unit tests (covering boundary values and invalid inputs) to empirically confirm correctness.

Tip 4: Problem Decomposition Before Code Generation

- **The Principle:** Directly prompting an AI to solve an entire multi-faceted project in one shot frequently leads to tangled, unmaintainable monolithic code.
- **The Practice:** Apply Computational Thinking: decompose the overarching problem into isolated modular components (e.g., input parser, data transformer, validation engine, output formatter). Prompt and verify each module sequentially, ensuring robust interfaces between them.

Tip 5: Strict Data Privacy & Security Hygiene

- **The Principle:** Anything uploaded to public cloud-hosted AI interfaces may be logged, analyzed, or incorporated into future training runs.
- **The Practice:** Never input sensitive credentials, API keys, database connection strings, personal identifying information (PII), or private repository tokens into AI prompts. Always use placeholder tokens (e.g., `<API_KEY_PLACEHOLDER>`) when debugging authenticated code.

---

4. Synthesis of Responsible AI Frameworks (VNPT AI Principles)

Analyzing the *Responsible AI* framework published by VNPT AI highlights five essential pillars that align directly with software engineering ethics:

Ethical Pillar	Meaning & Core Requirement	Practical Application in CS252
Transparency & Explainability	AI-generated reasoning and decision pathways must be inspectable, interpretable, and clearly communicated.	Documenting AI usage via transparent tables in `README.md`, citing prompts and human verification steps.
Human Agency & Accountability	Ultimate responsibility for automated outputs rests squarely on the human author/operator.	The student remains 100% accountable for all bugs, security flaws, or logical inconsistencies in submitted work.
Reliability, Safety & Robustness	Systems must operate predictably, resist failure, and degrade gracefully under edge conditions.	Stress-testing code with boundary conditions and sanitizing user inputs against erroneous or malicious data.
Fairness & Non-Discrimination	Models and datasets must avoid systemic bias against specific demographics or user groups.	In project design (e.g., our multimodal failure proposal), ensuring models perform reliably across diverse body types and physiological baselines.
Privacy & Data Governance	User data must be protected, confidential, and compliant with privacy regulations.	Refraining from transmitting private credentials or personal peer data to external AI model endpoints.

---

5. Academic Integrity Commitment

For this course (CS252 - Computational Thinking) and all subsequent computer science studies at HCMUS:

1. I commit to maintaining an explicit **AI Usage Declaration** in all submissions, disclosing tools utilized and the exact nature of their assistance.
2. I will actively cultivate independent problem-solving skills, utilizing AI strictly as a collaborative tutor and accelerator rather than an intellectual substitute.
3. I take complete ownership of my submissions, guaranteeing that every delivered concept, line of code, and architectural plan is verified, validated, and genuinely understood.